

GPU 计算与原生核心

Rust 与 WebAssembly · WebGPU · 三级降级

Ergalics Studio 的计算加速由两层构成：Rust 编译的 WebAssembly 原生核心（native/ergalics-core）提供设备管理与计算内核的参考实现；宿主侧的计算服务（src/core/gpu.ts 与 src/core/compute.ts）把这套能力以统一接口暴露给插件，并在 WASM 或 WebGPU 缺失时逐级降级到纯 CPU 路径。本篇覆盖五个 Rust 模块、可复用 WGSL 内核清单、两种内核调用路径与端到端数值验证。

三级计算路径

参考引擎

WASM 已加载
Rust 原生核心

加速路径

WebGPU 可用
GPU 内核

兜底路径

两者皆缺
CPU 一致实现

数值验证

GPU 与 CPU
误差 $2e-6$ 内

目 录

- 一、Rust 原生核心 03
 - 1.1 五个模块的职责划分 03
 - 1.2 面向 JavaScript 的暴露面 04
- 二、宿主侧计算服务 05
- 三、可复用 WGSL 内核 06
 - 3.1 内核清单 06
 - 3.2 两种内核调用路径 07
- 四、端到端数值验证 08
- 五、构建与降级说明 08

阅读提示：本篇的关键结论是——GPU 计算不是演示性质的功能点，而是被单元测试与 端到端数值一致性校验双重兜底的真实计算路径；并且任何环境下都有行为一致的回退方案。

Rust 原生核心

1.1 五个模块的职责划分

native/ergalics-core 编译目标为 wasm32-unknown-unknown，经 wasm-bindgen 绑定到 src/native（构建产物，不入库）。WebGPU 绑定依赖 web-sys 的实验性 API，通过构建配置中的 rustflags 开启。Rust 源码按职责分为五个模块：

源文件	职责
lib.rs	对外导出与初始化
device.rs	GpuDeviceManager：适配器与设备获取，带 CPU 回退选项
buffer.rs	GpuBuffer：以显式 usage 掩码创建存储、只读与均匀缓冲，write 上传、read 经专用回读缓冲读回
compute.rs	KernelDescriptor 与 BindingDescriptor、内核编译、绑定组物化与执行
utils.rs	detect_file_kind 等辅助（基于魔数的文件类型检测）

把这部分能力放在 Rust 而不是纯 TypeScript 里，有两个直接收益：一是资源生命周期（适配器、设备、缓冲、回读缓冲）由强类型系统管住，错误在编译期暴露；二是同一份核心代码可以脱离浏览器演进——它本来就是面向 wasm32 目标的独立 crate。

1.2 面向 JavaScript 的暴露面

能力	说明
GpuDeviceManager	适配器与设备获取，带 CPU 回退选项
GpuBuffer	显式 usage 掩码的缓冲创建、上传与经回读缓冲的读取
KernelDescriptor 与 BindingDescriptor	描述计算内核与缓冲绑定 (uniform、storage、read-only-storage, 动态偏移与最小绑定尺寸)
ComputeKernel 编译	从绑定描述符构建真实的 GPUBindGroupLayout, 编译 WGSL 模块并创建管线
ComputeKernel bind_group	从保留的布局物化绑定组 (第 i 个缓冲对应第 i 个绑定)
ComputeKernel run	一次调用完成绑定组、dispatch 与提交; dispatch 方法留给宿主自管命令编码器
compilation_info	异步暴露 WGSL 编译诊断 (错误或警告加行列号)
detect_file_kind	基于魔数的文件类型检测, 供加载器使用

注意 run 与 dispatch 的分工: run 面向"一次性算完"的简单场景, 把绑定、提交封装成一次调用; dispatch 则把命令编码器留给宿主, 供需要多内核串联、精细控制提交时机的插件使用。两种粒度覆盖了从直方图聚合到 N-Body 乒乓积分的全部使用形态。

宿主侧计算服务

src/core/gpu.ts 持有适配器与设备生命周期，负责 CPU 回退与显存不足跟踪。其上的 src/core/compute.ts 是面向插件的计算面（即 PluginApi.gpu）：createBuffer、write、read、compileKernel、compilationInfo 与一次性 run。路由逻辑：

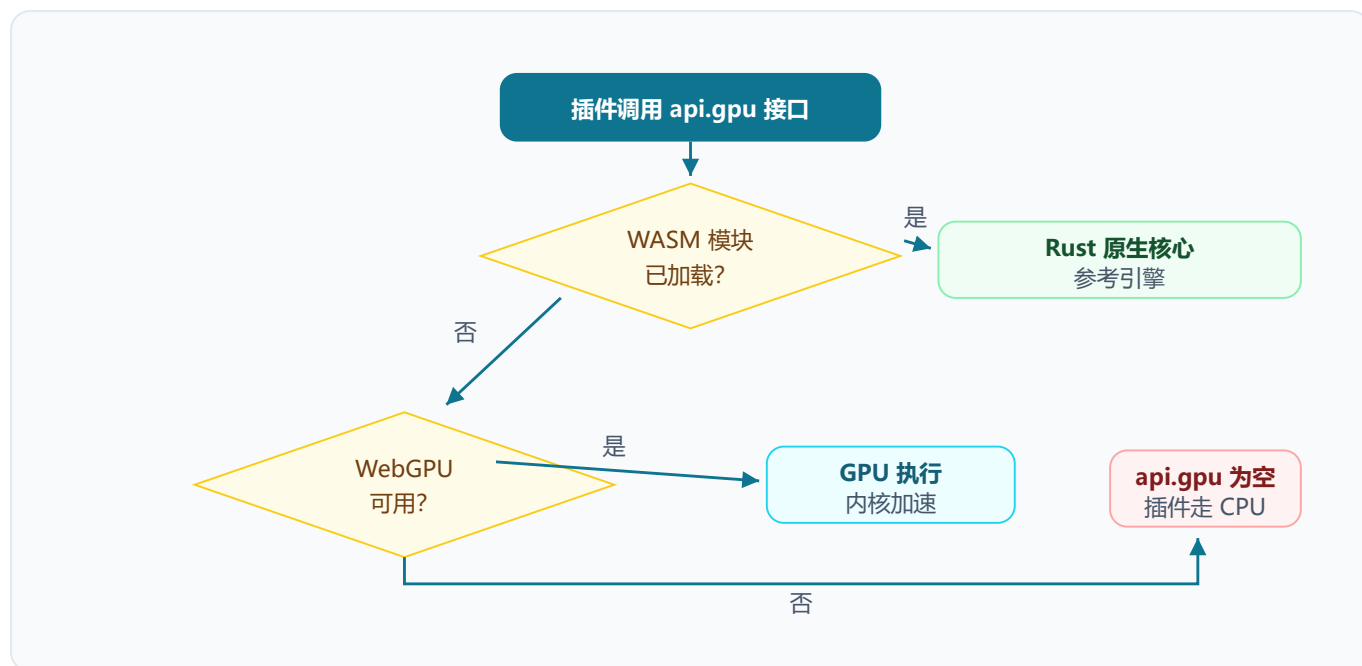


图 1 api.gpu 的计算路由：参考引擎优先，逐级降级到 CPU

这一设计保证开发与生产环境的加速计算均可用：Rust 核心始终是参考引擎，WebGPU 直连是加速路径，而每个内置插件的 CPU 回退跑的是与 GPU 内核数学一致的实现。路由决策对插件透明——插件只管调用 api.gpu，拿到的要么是能力面，要么是空值加明确的 CPU 回退指引。

可复用 WGSL 内核

3.1 内核清单

src/core/wgsl.ts 收纳可复用的 WGSL 计算内核，并配套与内核数学一致的宿主侧打包、解包辅助函数供 CPU 回退使用：

内核	数学内容	使用插件
粒子积分	交错式 [x, y, vx, vy] 单缓冲积分	粒子
三维全对引力	O(N 平方) 直接求和，乒乓缓冲避免逐步回读	N-Body 引力
D2Q9 碰撞与迁移	格子 Boltzmann 碰撞、迁移与涡量计算	流体模拟
波动方程 leapfrog	二维有限差分时间推进	波动方程
直方图、热力图、点云	数据聚合与投影加速	同名插件

3.2 两种内核调用路径

1. **单缓冲路径**（粒子演示）：上传交错式数据加均匀参数，dispatch WGSL 积分器，读回结果并上报真实 GPU 时间。
2. **乒乓缓冲路径**（N-Body 演示）：成对缓冲交替读写，每个积分步完全留在设备上，无逐步回读开销。

乒乓路径是理解本层设计的关键：若每步都把位置数据读回 CPU 再传回 GPU，数据往返的延迟会吞掉并行计算的收益。N-Body 把整条积分链留在设备内，只在渲染需要时做一次性读回——这是“计算必须是真计算”在数据搬运层面的具体化。

3.2 两种内核调用路径（N-Body 示例）



图 2 N-Body 乒乓缓冲路径：积分步完全留在设备上

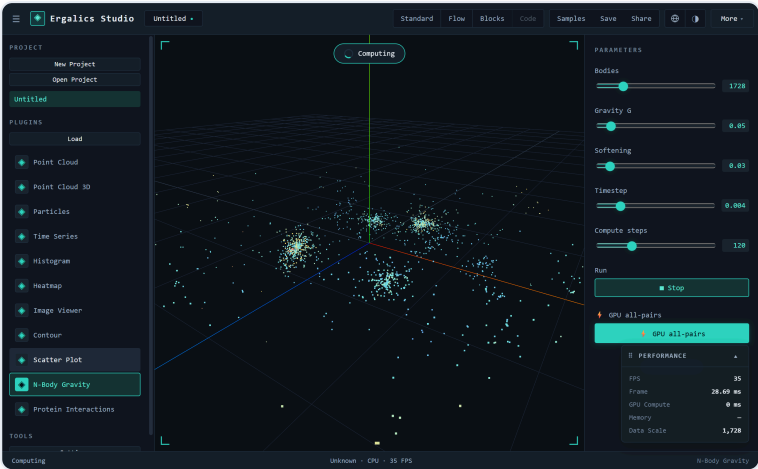


图 3 N-Body 引力：环形星环环绕中心质量（三维，WGSL 全对内核）

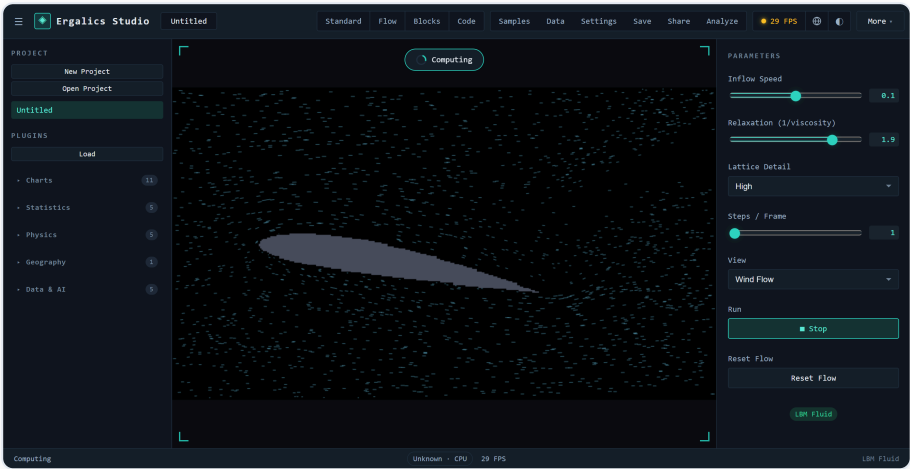


图 4 格子 Boltzmann 流体：绕机翼障碍物的通道流，D2Q9 双内核

四 端到端数值验证

GPU 路径不是摆设：verify-webgpu 端到端套件在无头 Edge（SwiftShader 软件渲染）中驱动真实 WebGPU 通路，用数值基准比较 GPU 结果与 CPU 积分器，误差要求在约 2 乘以 10 的负 6 次方以内；另有应用集成步骤点击粒子插件并断言出现 wasm 引擎的 GPU 提示。相关单元测试覆盖 WGSL 模板生成、参数打包、输出尺寸与 CPU 回退的一致性。

值得强调的是验证环境的严苛程度：无头浏览器里没有真实显卡，WebGPU 走的是 SwiftShader 软件渲染。能在软件渲染下保持数值一致，意味着校验的是内核数学本身而不是某个特定硬件的浮点行为，这套测试在任何持续集成环境都能复现。

五 构建与降级说明

构建命令链为：先执行 build:wasm 将 Rust 核心编译进 src/native，再进行类型检查与 Vite 生产构建。只构建前端可跳过 WASM 步骤（build:web）。持续集成环境不安装完整 Rust 工具链，由存根生成脚本提供占位 WASM 模块。

当 WASM 模块缺失时前端优雅降级：欢迎页硬件自检会如实报告 WebGPU、WASM 与 IndexedDB 的可用性；计算路由自动改走原生 WebGPU API 或 CPU 实现；插件在无 GPU 环境下以 CPU 完成同样的数学，行为一致。三级降级路径总结：

档位	条件	计算路径
参考引擎	WASM 模块已加载	Rust 原生核心
加速路径	WebGPU 可用（含经 WASM 或直连）	GPU 内核
兜底路径	两者皆缺	与内核数学一致的 CPU 实现

降级不是“功能缩水”的同义词：三条路径跑的是同一套数学，差别只在速度。这正是每一个 GPU 内核都配一个 CPU 孪生实现的原因——用户在任何设备上看到的结果都一致。